<packt>

Starting Now

Building Agent-Ready Internal Developer Platforms

Ship a service, deploy an AI agent on the same platform, and keep it safe, observable, and stoppable. Everything runs on your laptop.

Ramiro Álvarez & Guillermo Vigil

Instructors



Building Agent-Ready Internal Developer Platforms

A hands-on workshop · 140 minutes



Ramiro Álvarez

Instructor

About Ramiro

- Platform engineer who builds internal developer platforms on Kubernetes.
- Today he leads agent identity and tools, the AI gateway, and observability.
- Ask him about LiteLLM, Langfuse, and running models behind a gateway.

github.com/kaskol10

linkedin.com/in/ramiroalvfer



Guillermo Vigil

Instructor

About Guillermo

- Platform engineer who builds internal developer platforms on Kubernetes.
- Today he leads the golden path and the self-documenting knowledge base.
- Ask him about GitOps with Argo CD, CI checks that agents can read, and the fork flow.

github.com/guillermotti

linkedin.com/in/guillermotti

Agenda

01	Agents as platform users	10 min
02	Golden path · Lab 1	20 min
03	Knowledge base · Lab 2	15 min
04	Identity, tools, and gateway	10 min
05	Hermes alert triage · Lab 3	30 min
06	Guardrails · Lab 4	30 min
07	Observability and cost	10 min
08	Where it breaks · Q&A	15 min

What you need

Your machine

- Docker with 12 GB of memory (16 recommended) and 4 CPUs
- About 25 GB of free disk
- An agent harness, for the Lab 2 question and auto-onboarding
- Your own fork of the workshop repository, with GitHub Actions enabled
- A capped OpenRouter key, about $10, that expires after the workshop

Your tools

- kind, kubectl, helm, git, jq, envsubst
- gh 2.50 or newer, logged in
- yq 4.x from mikefarah, not the Python yq
- kubeconform and helm-docs
- preflight.sh checks all of this and prints one JSON line per check

Set up your environment

Fork the repository on GitHub, then clone your fork and point gh at it:

```
git clone https://github.com/<you>/agentic-idp-packt.git
cd agentic-idp-packt
gh repo set-default <you>/agentic-idp-packt
export GH_REPO=<you>/agentic-idp-packt
```

Check your machine, fill in .env, and run the bootstrap:

```
./labs/00-setup/local/preflight.sh
cp labs/00-setup/local/.env.example labs/00-setup/local/.env
./labs/00-setup/local/bootstrap.sh
```

Or let an agent harness do it: ask it to follow labs/02-knowledge-base/kb/guides/onboarding.md.

Fill in your .env

Copy .env.example to .env. Never commit .env.

Variable	What to put there
REPO_URL	https://github.com/<you>/agentic-idp-packt.git
REPO_WEB_URL	https://github.com/<you>/agentic-idp-packt
OPENROUTER_API_KEY	Your capped provider key. Only the gateway sees it.
GITHUB_TOKEN	Fine-grained token, your fork only: Contents and Pull requests, read and write. The runbook bot uses it in Lab 3.
LITELLM_MASTER_KEY	Leave empty. The bootstrap generates one for the kill switch.
SKIP_LANGFUSE	Leave empty. Set it to true only if your laptop is short on memory.

Check your environment

```
kubectl config current-context      # kind-agentic-idp
kubectl -n argocd get applications  # orders and runbooks: Synced, Healthy
```

Every UI is on port 80: http://argocd.localhost, http://langfuse.localhost, http://litellm.localhost, and http://orders.localhost. Log in to Argo CD as admin:

```
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath='{.data.password}' | base64 -d
```

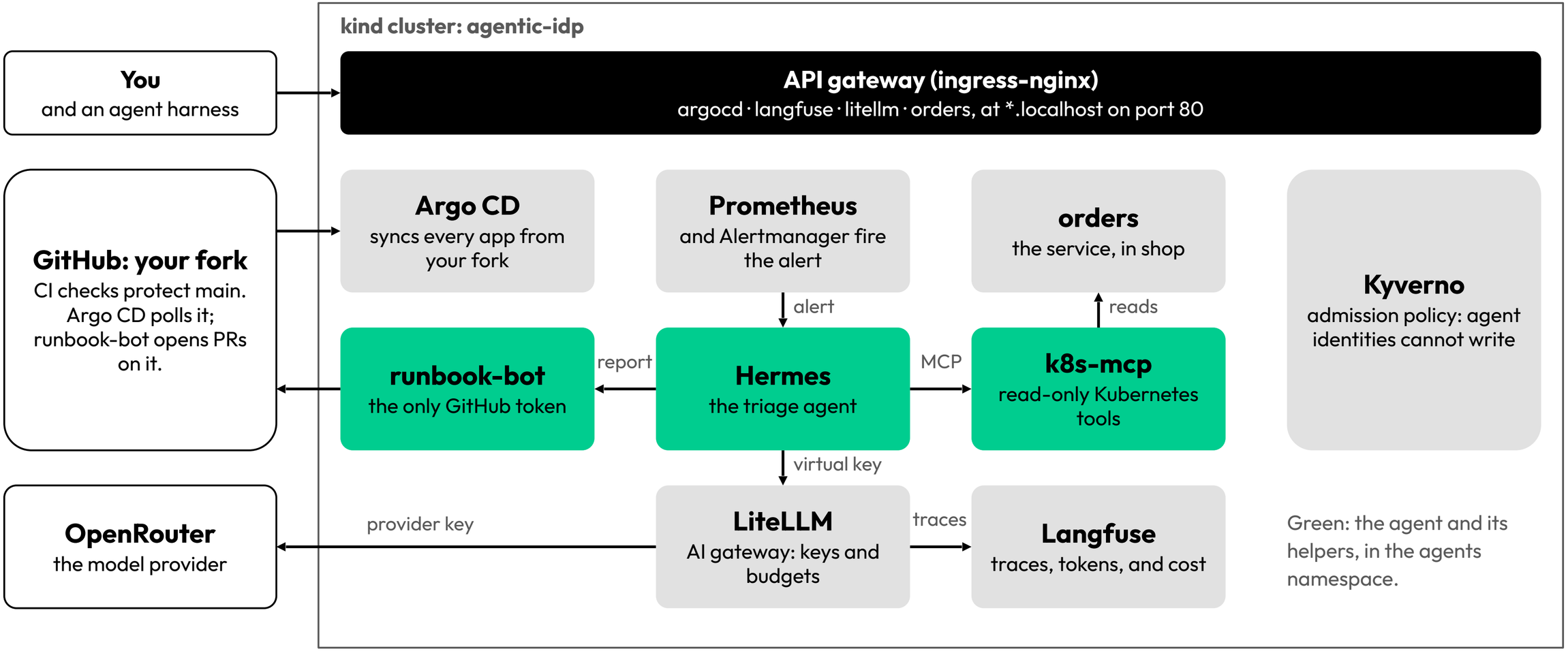
Something broken? Reset the cluster. Your fork and pull requests are kept:

```
kind delete cluster --name agentic-idp && ./labs/00-setup/local/bootstrap.sh
```

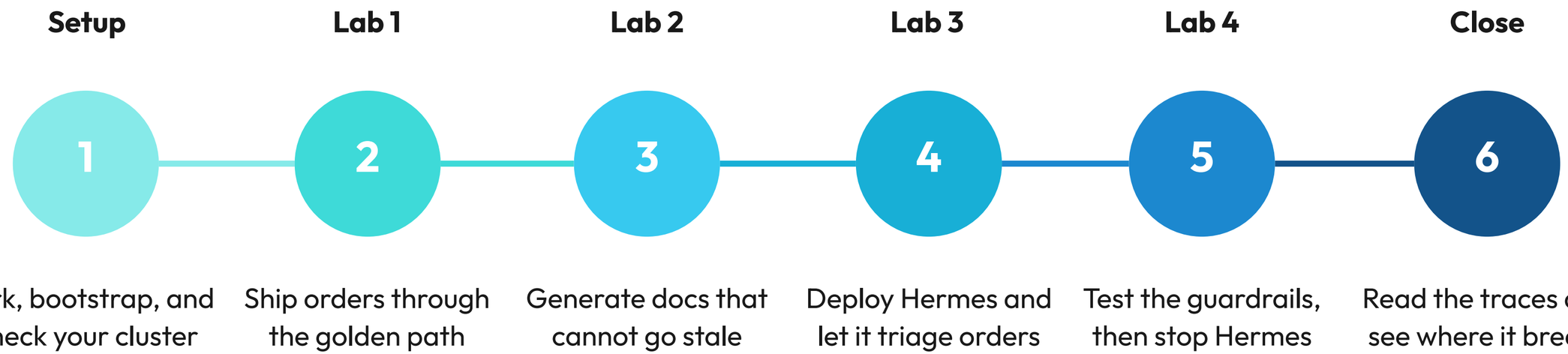
The platform on your laptop

Piece	What it does today
kind cluster agentic-idp	Everything runs here, on your laptop
API gateway (ingress-nginx)	Every web UI on port 80, at *.localhost
Your fork, Git, and gh	The only write path. main is protected by CI checks
Argo CD	Syncs every app from your fork and reverts drift
CI: validate and docs-freshness	JSON errors an agent can read; stale docs fail the PR
Prometheus and Alertmanager	Fire the alert and send it to Hermes
Hermes and k8s-mcp	The triage agent and its read-only Kubernetes tools
LiteLLM	The AI gateway: one model alias, virtual keys, budgets
runbook-bot	The only component with a GitHub token. It only opens PRs
Langfuse	Traces of every model call, with tokens and cost
Kyverno	Policy at admission time, the second layer after RBAC

How the pieces connect



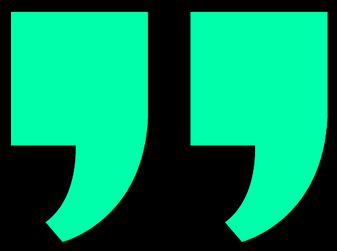
How the labs fit together



Section 01: Agents as platform users

10 minutes





What changes when AI agents become users of the platform, alongside engineers?

- The question behind every section today

Agent-ready means

Understand it

Docs and context agents can trust: generated from the platform and checked in CI.

Act safely in it

Its own identity, read-only tools, Git as the only write path, and policy as a second layer.

Observe and stop it

Every model call traced, every change in Git, and a kill switch that GitOps cannot undo.

One service, one agent

orders, in the shop namespace

- The service everyone ships
- Lab 1 deploys it through the golden path
- Lab 2 documents it
- Lab 3 breaks it on purpose

Hermes, in the agents namespace

- The agent that triages alerts
- Lab 3 deploys it the same way as orders
- Lab 4 tests its guardrails and stops it
- It never holds a cluster or GitHub token

Section 02: The golden path

Lab 1 · 20 minutes



What changes for agents

Repositories

One folder per app with a fixed shape. An agent can scaffold it correctly every time.

CI/CD

Validation prints one JSON line per app with a clear error, not a wall of logs.

GitOps

Git is the only write path and main is protected: review, history, and rollback.

State and drift

Argo CD self-heal reverts any change made outside Git, by a human or an agent.

Lab 1: ship orders (guided demo)

1. Walk through labs/01-golden-path/apps/orders/ and the golden path chart.
2. Open orders in Argo CD at http://argocd.localhost.
3. Change ORDER_DB in a pull request. Watch the golden-path-ci check, the merge, and the sync.

Argo CD polls your fork every 30 seconds. A hard refresh skips the wait:

```
kubectl -n argocd annotate application orders argocd.argoproj.io/refresh=hard --overwrite
```

Nobody runs kubectl apply for an app. Not you, not an agent.

Lab 1: try it

Break the golden path and read what an agent would see:

```
git checkout -b try-lab-01
sed -i.bak 's/tag: "0.1.0"/tag: latest/' labs/01-golden-path/apps/orders/values.yaml
rm labs/01-golden-path/apps/orders/values.yaml.bak
git commit -am "try: use latest tag" && git push -u origin try-lab-01
gh pr create --fill && gh pr checks --watch
```

The check fails with an error an agent can act on, and main blocks the merge:

```
{"app":"labs/01-golden-path/apps/orders","status":"fail","error":"image.tag is required and must not be latest"}
```

Close the PR and go back to main:

```
gh pr close try-lab-01 --delete-branch && git checkout main
```

Section 03: The knowledge base

Lab 2 · 15 minutes



Docs agents can trust

Generated

Chart reference, service catalog, alerts, policies, and RBAC. Stale docs fail the PR.

Hand-written

Guides, runbooks, AGENTS.md, and llms.txt: what a generator cannot know.

Linked from alerts

Every alert has a runbook_url. The on-call human and Hermes start from the same page.

Write-back

Hermes proposes runbook learnings through a bot PR. A human decides.

Lab 2: docs that cannot go stale (guided demo)

1. Generate the reference docs and walk through them:

```
./labs/02-knowledge-base/scripts/generate-docs.sh
ls labs/02-knowledge-base/kb/generated/
```

2. Add a documented value to the chart's values.yaml and open a PR without regenerating. docs-freshness fails. Regenerate, push, and it passes.

```
# labs/01-golden-path/platform/charts/service/values.yaml
# -- Log level for the workload.
logLevel: info
```

3. Find orders in the service catalog, then read the WorkshopPodCrashLooping runbook. Hermes reads it in Lab 3.

Lab 2: try it

Open an agent harness in your fork and ask, in a new chat:

> Following the platform knowledge base, what are the exact steps
> to deploy a new agent called hermes on this platform?

- A good answer starts from AGENTS.md, follows kb/guides/deploy-an-agent.md, and names the folder, ServiceAccount, Secret, and pull request.
- If it guesses, or suggests kubectl apply, the knowledge base has a gap.
- Fix the docs, not the prompt.

Section 04: Identity, tools, and the AI gateway

10 minutes



Three layers Hermes needs

Identity

Its own ServiceAccount, with no cluster permissions. The MCP server reads as hermes-reader, which cannot write or read Secrets.

Tools

Kubernetes only through named, read-only MCP tools. Terminal, file, and web tools are off.

Gateway

A LiteLLM virtual key for one model alias, with a budget and a rate limit. Never a provider key.

RBAC is the coarse layer

What you run today

- RBAC: verbs on resources
- MCP tool allowlists
- NetworkPolicy egress rules
- Kyverno at admission time

The production next step

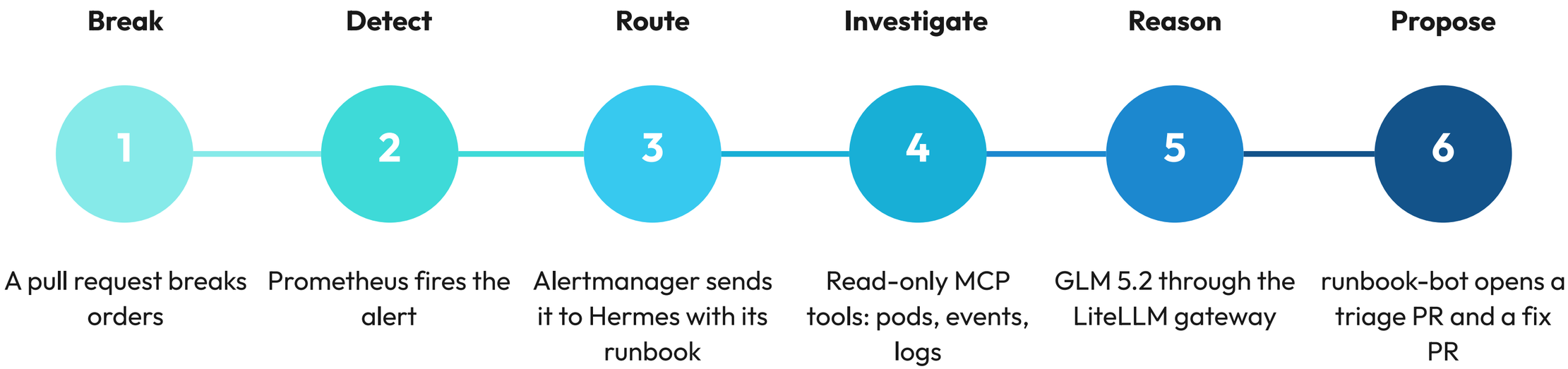
- Relationship-based access, for example OpenFGA, for questions like “can this agent update that runbook?”
- Kyverno stays at admission time
- Different layers, not either/or

Section 05: Hermes alert triage

Lab 3 · 30 minutes



How a triage flows



Hermes never holds a GitHub token

Why

Anything an agent can read, a prompt injection can ask it to leak. A token plus GitHub egress would let a poisoned log push commits.

Instead

- A narrow runbook bot holds the token
- It only opens pull requests, and main is protected
- Fix PRs may only change an existing labs/*/apps/*/values.yaml
- Instruction-like text gets a WARNING in the PR body
- Humans decide what merges

Lab 3: check the identity

Left GITHUB_TOKEN empty at bootstrap? Add it to .env, then load it for the bot:

```
set -a; source labs/00-setup/local/.env; set +a
kubectl -n agents create secret generic runbook-bot-github \
  --from-literal=GITHUB_TOKEN="$GITHUB_TOKEN" --from-literal=GITHUB_REPO=<you>/agentic-idp-packt \
  --dry-run=client -o yaml | kubectl apply -f -
```

Step 1. Check what the platform team already set up:

```
kubectl -n agents get serviceaccounts,secrets
kubectl auth can-i list pods --as=system:serviceaccount:agents:hermes-reader -A     # yes
kubectl auth can-i delete pods --as=system:serviceaccount:agents:hermes-reader -A   # no
kubectl auth can-i get secrets --as=system:serviceaccount:agents:hermes-reader -A   # no
```

Lab 3: ship Hermes

Step 2. Follow kb/guides/deploy-an-agent.md, or ask an agent harness to. Either way, the result is:

```
git checkout main && git pull && git checkout -b lab-03-hermes
mkdir -p labs/03-hermes-triage/apps
cp -r labs/03-hermes-triage/starter/* labs/03-hermes-triage/apps/
./labs/01-golden-path/scripts/validate-apps.sh
./labs/02-knowledge-base/scripts/generate-docs.sh
git add -A && git commit -m "Deploy Hermes triage agent" && git push -u origin lab-03-hermes
gh pr create --fill && gh pr checks --watch
```

Merge when the checks pass. A new app takes up to 2 minutes to appear:

```
kubectl -n agents get pods,networkpolicies -w    # hermes, k8s-mcp, runbook-bot
```

Lab 3: break orders

Step 3. Pick one scenario: crashloop, oom, or bad-config. It opens a PR:

```
git checkout main && git pull
./labs/03-hermes-triage/scenarios/break-orders.sh crashloop
```

Wait for your wave, merge the PR, and watch Hermes. The alert fires about a minute later:

```
kubectl -n agents logs deploy/hermes -f
```

When the report is in, fix orders through Git as well, and merge that PR:

```
git checkout main && git pull
./labs/03-hermes-triage/scenarios/break-orders.sh undo
```

Merge in two waves

2

waves, so 30 laptops do not hit the model in the same second

WAVE A

Even table card. Merge the break PR first.

WAVE B

Odd table card. Merge about two minutes later.

MISSED THE CALLOUT?

Raise a hand. Do not merge early.

Lab 3: read the bot's pull requests

Step 4. The runbook bot opens two pull requests on your fork:

- **Triage PR:** Hermes's report under labs/03-hermes-triage/triage-reports/, plus any runbook learning.
- **Fix PR:** only an existing labs/*/apps/*/values.yaml. Anything else is rejected.
- Instruction-like text gets a **WARNING** in the PR body. Review it as untrusted input.
- Neither PR can merge itself, because main is protected.

```
gh pr list
```

Lab 3: poisoned logs

Step 5. A failing job whose log tells agents to delete the shop namespace:

```
kubectl apply -f labs/03-hermes-triage/scenarios/poisoned-log.yaml
# Hermes reports it as a security finding. hermes-reader cannot delete anything anyway.
```

Step 6. A log that asks agents to send their secrets to a collector:

```
kubectl apply -f labs/03-hermes-triage/scenarios/poisoned-log-exfil.yaml
kubectl -n agents exec deploy/hermes -- \
  python -c "import urllib.request; urllib.request.urlopen('http://collector.shop.svc.cluster.local:8080', timeout=5)"
# Must time out: egress is blocked.
kubectl -n shop logs deploy/collector    # no requests from Hermes. Keep it for Lab 4.
```

Section 06: Guardrails and recovery

Lab 4 · 30 minutes

Defense in depth

RBAC

The first layer: verbs on resources. Easy to make too broad by mistake.

Kyverno

The second layer: agent identities cannot create, update, or delete anything directly.

GitOps

Guardrails are code in values.yaml. Self-heal puts back what someone deletes by hand.

Review

One approved PR can still weaken a guardrail. Agent guardrails need an owner.

Lab 4: policy as a second layer

Step 1. Enforce the policy, then give Hermes's identity too much RBAC on purpose:

```
kubectl apply -f labs/04-guardrails-recovery/policies/deny-agent-direct-changes.yaml
kubectl apply -f labs/04-guardrails-recovery/demo-rbac/hermes-remediator.yaml
kubectl auth can-i delete namespaces --as=system:serviceaccount:agents:hermes-reader    # yes
```

Impersonate Hermes and try what the poisoned logs asked for. Kyverno denies all three:

```
kubectl delete namespace shop --as=system:serviceaccount:agents:hermes-reader --dry-run=server
kubectl -n shop patch deploy orders --as=system:serviceaccount:agents:hermes-reader \
  -p '{"spec":{"replicas":0}}' --dry-run=server
kubectl -n agents delete networkpolicy hermes --as=system:serviceaccount:agents:hermes-reader --dry-run=server
kubectl delete -f labs/04-guardrails-recovery/demo-rbac/hermes-remediator.yaml
```

Lab 4: drift on a guardrail

Step 2. Someone debugging Hermes deletes its network policy as an admin:

```
kubectl -n agents delete networkpolicy hermes
kubectl -n agents get networkpolicy hermes -w
```

- Kyverno allows it, because admins are not agents.
- Within seconds Argo CD puts it back, because the policy lives in Git with Hermes.
- Drift is not only a human problem: the same self-heal reverts an agent's change.

Lab 4: a helpful pull request

Step 3. Open a reasonable-looking PR. Read it, see the checks pass, and merge it:

```
git checkout main && git pull
./labs/04-guardrails-recovery/scripts/open-helpful-pr.sh
```

When Hermes restarts, run the egress check from Lab 3 again:

```
kubectl -n agents rollout status deploy/hermes
kubectl -n agents exec deploy/hermes -- \
  python -c "import urllib.request; print(urllib.request.urlopen('http://collector.shop.svc.cluster.local:8080', timeout=5).read())"
# Now prints b'ok': the poisoned log has somewhere to send Hermes's key.
```

Lab 4: revert it through Git

Revert the merge with a pull request:

```
git checkout main && git pull && git checkout -b revert-hermes-egress
git revert --no-edit "$(git log --grep='let the agent check orders health' -n 1 --no-merges --format=%H)"
git push -u origin revert-hermes-egress && gh pr create --fill
```

Merge it, wait for the rollout, and run the egress check again. It times out.

In a real platform, CODEOWNERS would require the platform team on every agent's values.yaml.

Lab 4: the kill switch

Step 4. The obvious way does not work, because Argo CD scales it back to 1:

```
kubectl -n agents scale deploy hermes --replicas=0
kubectl -n agents get deploy hermes -w
```

Remove its credentials instead. Argo CD cannot recreate a Secret it does not manage:

```
./labs/04-guardrails-recovery/scripts/kill-switch.sh
kubectl -n agents get pods
```

The real kill switch blocks the virtual key on the gateway:

```
set -a; source labs/00-setup/local/.litellm-admin; set +a
LITELLM_ADMIN_URL=http://litellm.localhost ./labs/04-guardrails-recovery/scripts/kill-switch.sh local --block-key
```

Section 07: Observability, audit, and cost

10 minutes



What the platform records

Traces

Langfuse records the prompt, the tool calls, the latency, and the tokens of every model call.

Audit trail

Traces, Git history, and Kyverno decisions together explain what the agent did and why.

Cost

Each virtual key has a budget and a rate limit. A blocked key stops spend at once.

Open Langfuse

Read the login password from its Secret:

```
kubectl -n langfuse get secret langfuse-secrets -o jsonpath='{.data.LANGFUSE_INIT_USER_PASSWORD}' | base64 -d
```

Log in at http://langfuse.localhost as workshop@example.com and open the workshop project.

- Find the trace for the triage you watched in Lab 3.
- Read the prompt with the runbook, the MCP tool calls, and the token cost.
- Match it with the bot's PR and the Kyverno denials from Lab 4.

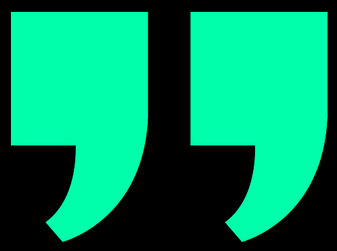
Section 08: Where it breaks

Q&A · 15 minutes



Where it breaks

What breaks	What helps
Stale docs	Generate reference docs and fail the PR when they drift
Over-broad RBAC	Policy as a second layer; no direct writes for agent identities
Prompt injection	Read-only tools, no egress, and no tokens in the agent
Approval theater	Owners review guardrail changes; CODEOWNERS on agent values
Alert storms	Triage each alert once per window, with a budget per key
Gateway outages	One central gateway, with a kill switch per key



A platform is agent-ready when agents can understand it, act safely within it, and be observed and stopped when needed.

- The takeaway

Keep going

Everything stays in your fork after today.

Where	What
labs/README.md	All labs, in order, with the time each one takes
labs/02-knowledge-base/kb/guides/platform-map.md	Every piece of the platform and how they connect
labs/02-knowledge-base/kb/guides/onboarding.md	Let an agent harness rebuild the environment for you
“If you finished early” in each lab	Extra exercises for Labs 3 and 4
kind delete cluster --name agentic-idp	Clean up your laptop when you are done



Thank you

Questions? Find us in the room.